CODE / PRACTISE / IMPROVE

PYTHON

PRACTICE WORKBOOK

Beginner → Intermediate

60 CHALLENGES / 12 CHAPTERS / PYTHON 3.10+

PART I — EXERCISES | PART II — WORKED SOLUTIONS

WHAT YOU WILL PRACTISE

Core syntax • Strings • Conditions • Loops • Collections • Functions

Comprehensions • Exceptions • Files • CSV/JSON • OOP • Generators

Built for active learning. Write code first, run your own tests, then compare with the explanation.

BEFORE YOU BEGIN

Solve exercises in order or choose a topic you want to strengthen. Each challenge is independent unless stated otherwise (exercise 56 builds on exercise 51). Use Python 3.10 or later; no third-party packages are needed.

1 Read the problem and example carefully. Decide what the function or class must return.

2 Code without consulting the answer key. Try at least two additional test cases.

3 Check edge cases: zero, empty input, duplicates, and invalid values when applicable.

4 Mark each problem: [ ] Not started [~] Needs review [x] Confident.

5 Only then compare your answer with the separate worked solution and explanation.

CHAPTER CHECKLIST

#	TOPIC	EXERCISES	STATUS
01	Python Basics & Numbers	01–05	[ ]
02	Working with Strings	06–10	[ ]
03	Decisions & Boolean Logic	11–15	[ ]
04	Loops & Number Patterns	16–20	[ ]
05	Lists & Tuples	21–25	[ ]
06	Dictionaries & Sets	26–30	[ ]
07	Functions & Reuse	31–35	[ ]
08	Comprehensions & Sorting	36–40	[ ]
09	Errors, Tests & Defensive Code	41–45	[ ]
10	Files, CSV & JSON	46–50	[ ]
11	Python Classes & OOP	51–55	[ ]
12	Intermediate Patterns & Mini Projects	56–60	[ ]

SELF-CHECK

At the end of a chapter, ask: Can I solve a similar problem without looking anything up? Can I explain why my solution works? Can I identify one edge case?

PART I / EXERCISES

CHAPTER 01 / BEGINNER

Python Basics & Numbers

Variables, arithmetic, type conversion, formatting

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

01 Convert Celsius to Fahrenheit Easy • ~8 min

Write `celsius_to_fahrenheit(celsius)` that returns a float using F = (C × 9/5) + 32. Round the result to 1 decimal place.

SAMPLE INPUT / OUTPUT

```
celsius_to_fahrenheit(0) -> 32.0
celsius_to_fahrenheit(37) -> 98.6
```

02 Split a restaurant bill Easy • ~8 min

Write `split_bill(total, tip_percent, people)` returning the amount each person pays, rounded to 2 decimals. Add a percentage tip to `total`. Assume `people > 0`.

SAMPLE INPUT / OUTPUT

```
split_bill(1200, 10, 3) -> 440.0
split_bill(500, 0, 2) -> 250.0
```

03 Seconds into hours, minutes, seconds Easy • ~8 min

Write `format_duration(seconds)` for non-negative integer seconds. Return a string formatted `HH:MM:SS`, with at least two digits per part. Hours may exceed 99.

SAMPLE INPUT / OUTPUT

```
format_duration(3661) -> '01:01:01'
format_duration(61) -> '00:01:01'
```

04 Calculate compound growth Easy • ~8 min

Write `future_value(principal, annual_rate_percent, years)` for annual compounding. Return the future amount rounded to 2 decimals. Assume non-negative years.

SAMPLE INPUT / OUTPUT

```
future_value(1000, 10, 2) -> 1210.0
future_value(500, 0, 4) -> 500.0
```

05 Calculate a weighted average Easy • ~8 min

Write `weighted_average(scores, weights)` that returns the weighted average rounded to 2 decimals. Lists have the same positive length, and the weights add to a positive number.

SAMPLE INPUT / OUTPUT

```
weighted_average([80, 90], [1, 3]) -> 87.5
weighted_average([10, 20, 30], [1, 1, 2]) -> 22.5
```

CHAPTER 02 / BEGINNER

Working with Strings

Slicing, built-in string methods, normalization

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

06 Clean a full name Easy • ~8 min

Write `clean_name(name)` that strips extra spaces from the ends, reduces multiple internal spaces to one, and capitalizes the first letter of each word.

SAMPLE INPUT / OUTPUT

```
clean_name('  aLIce   jONES  ') -> 'Alice Jones'
clean_name('MARY jane') -> 'Mary Jane'
```

07 Reverse words, not letters Easy • ~8 min

Write `reverse_words(sentence)` that returns the words in reverse order, separated by single spaces. Treat any whitespace as a separator.

SAMPLE INPUT / OUTPUT

```
reverse_words('Python is fun') -> 'fun is Python'
reverse_words(' a   b ') -> 'b a'
```

08 Check a relaxed palindrome Easy • ~8 min

Write `is_palindrome(text)` that ignores case and all non-alphanumeric characters, then checks whether the cleaned text reads the same backward.

SAMPLE INPUT / OUTPUT

```
is_palindrome('A man, a plan, a canal: Panama!') -> True
is_palindrome('Python') -> False
```

09 Count vowels Easy • ~8 min

Write `count_vowels(text)` that counts A, E, I, O, U in any case. Do not count Y.

SAMPLE INPUT / OUTPUT

```
count_vowels('Data Engineering') -> 7
count_vowels('rhythm') -> 0
```

10 Mask an email username Easy • ~8 min

Write `mask_email(email)` for a valid email with at least one character before `@`. Show the first character of the username and replace the remaining username characters with `*`. Preserve the domain.

SAMPLE INPUT / OUTPUT

```
mask_email('alice@example.com') -> 'a****@example.com'
mask_email('a@x.io') -> 'a@x.io'
```

CHAPTER 03 / BEGINNER

Decisions & Boolean Logic

if / elif / else, comparisons, validation

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

11 Classify a number Easy • ~8 min

Write `classify_number(n)` returning one of ``positive even'`, ``positive odd'`, ``negative even'`, ``negative odd'`, or ``zero'`.

SAMPLE INPUT / OUTPUT

```
classify_number(-4) -> 'negative even'
classify_number(0) -> 'zero'
```

12 Leap year rule Easy • ~8 min

Write `is_leap_year(year)` using Gregorian calendar rules: divisible by 4, except centuries must also be divisible by 400.

SAMPLE INPUT / OUTPUT

```
is_leap_year(2000) -> True
is_leap_year(1900) -> False
is_leap_year(2024) -> True
```

13 Shipping fee bands Easy • ~8 min

Write `shipping_fee(order_total)` using: free for orders ≥ 1500; 80 for orders ≥ 500 but < 1500; 150 for orders < 500. Assume non-negative order totals.

SAMPLE INPUT / OUTPUT

```
shipping_fee(2000) -> 0
shipping_fee(500) -> 80
shipping_fee(499) -> 150
```

14 Validate a triangle Easy • ~8 min

Write `triangle_type(a, b, c)`. Return ``invalid'` unless all sides are positive and satisfy the strict triangle inequality. Otherwise return ``equilateral'`, ``isosceles'`, or ``scalene'`.

SAMPLE INPUT / OUTPUT

```
triangle_type(3, 3, 3) -> 'equilateral'
triangle_type(3, 4, 5) -> 'scalene'
triangle_type(1, 2, 3) -> 'invalid'
```

15 Grade a score Easy • ~8 min

Write `grade(score)` returning `'A'` for 90–100, `'B'` for 80–89, `'C'` for 70–79, `'D'` for 60–69, `'F'` for 0–59. Return `'Invalid'` for scores outside 0–100.

SAMPLE INPUT / OUTPUT

```
grade(89) -> 'B'
grade(105) -> 'Invalid'
```

CHAPTER 04 / BEGINNER

Loops & Number Patterns

for, while, range, simple algorithms

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

16 Sum multiples of 3 or 5 Easy • ~8 min

Write `sum_multiples(limit)` summing integers from 1 up to but excluding `limit` that are divisible by 3 or 5. Assume `limit >= 0`.

SAMPLE INPUT / OUTPUT

```
sum_multiples(10) -> 23
sum_multiples(1) -> 0
```

17 Factorial with a loop Easy • ~8 min

Write `factorial(n)` returning n! for a non-negative integer n. Remember that 0! = 1.

SAMPLE INPUT / OUTPUT

```
factorial(5) -> 120
factorial(0) -> 1
```

18 FizzBuzz list Easy • ~8 min

Write `fizzbuzz(n)` returning a list for 1 through n. Replace multiples of 3 with `'Fizz'`, of 5 with `'Buzz'`, and of both with `'FizzBuzz'`. Keep all other values as integers.

SAMPLE INPUT / OUTPUT

```
fizzbuzz(5) -> [1, 2, 'Fizz', 4, 'Buzz']
fizzbuzz(0) -> []
```

19 Count digits without strings Easy • ~8 min

Write `digit_count(n)` returning the number of decimal digits of an integer, including negative values. Do not convert to a string. Zero has 1 digit.

SAMPLE INPUT / OUTPUT

```
digit_count(-12345) -> 5
digit_count(0) -> 1
```

20 Find all primes up to n Easy • ~8 min

Write `primes_up_to(n)` returning all prime numbers ≤ n in ascending order. A prime is an integer greater than 1 divisible only by 1 and itself.

SAMPLE INPUT / OUTPUT

```
primes_up_to(10) -> [2, 3, 5, 7]
primes_up_to(1) -> []
```

CHAPTER 05 / BEGINNER+

Lists & Tuples

Indexing, mutation, traversal, nested lists

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

21 Remove duplicates, keep order Medium • ~12 min

Write `unique_in_order(items)` that returns a new list with only the first occurrence of each value. Assume all values are hashable. Do not mutate the input.

SAMPLE INPUT / OUTPUT

```
unique_in_order([3, 1, 3, 2, 1]) -> [3, 1, 2]
unique_in_order([]) -> []
```

22 Second distinct largest Medium • ~12 min

Write `second_largest(values)` returning the second-largest distinct number, or `None` if fewer than two distinct values exist.

SAMPLE INPUT / OUTPUT

```
second_largest([4, 1, 4, 3]) -> 3
second_largest([8, 8]) -> None
```

23 Rotate a list right Medium • ~12 min

Write `rotate_right(items, k)` returning a new list rotated right by k positions. Assume `k >= 0`; do not mutate the input. Empty lists should return `[]`.

SAMPLE INPUT / OUTPUT

```
rotate_right([1, 2, 3, 4], 1) -> [4, 1, 2, 3]
rotate_right([1, 2], 5) -> [2, 1]
```

24 Running totals Medium • ~12 min

Write `running_totals(numbers)` returning a list where item i is the sum of the first i+1 numbers. Empty input yields an empty list.

SAMPLE INPUT / OUTPUT

```
running_totals([2, -1, 5]) -> [2, 1, 6]
running_totals([]) -> []
```

25 Transpose a rectangular matrix Medium • ~12 min

Write `transpose(matrix)` for a rectangular list of lists. Return rows as columns; if input is empty or contains empty rows, return `[]`. Do not mutate the matrix.

SAMPLE INPUT / OUTPUT

```
transpose([[1, 2, 3], [4, 5, 6]]) -> [[1, 4], [2, 5], [3, 6]]
transpose([]) -> []
```

CHAPTER 06 / BEGINNER+

Dictionaries & Sets

Counting, grouping, keys, membership

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

26 Word frequency counter Medium • ~12 min

Write `word_counts(text)` returning a dict of counts for whitespace-separated words. Ignore case and strip leading/trailing `.,!?;:` from each word. Skip empty results.

SAMPLE INPUT / OUTPUT

```
word_counts('Hi, hi! Python.') -> {'hi': 2, 'python': 1}
word_counts('!?') -> {}
```

27 Merge inventory counts Medium • ~12 min

Write `merge_inventory(left, right)` returning a new dictionary. If a product appears in both dictionaries, add quantities. Do not change either input.

SAMPLE INPUT / OUTPUT

```
merge_inventory({'pen': 2}, {'pen': 3, 'book': 1}) -> {'pen': 5, 'book': 1}
```

28 Common unique values Medium • ~12 min

Write `common_values(a, b)` returning a sorted list of distinct values that occur in both lists. Assume values are comparable and hashable.

SAMPLE INPUT / OUTPUT

```
common_values([3, 2, 3, 1], [2, 4, 3]) -> [2, 3]
common_values([], [1]) -> []
```

29 Group names by initial Medium • ~12 min

Write `group_by_initial(names)` returning a dictionary mapping each uppercase initial to a list of original names in input order. Ignore empty or whitespace-only entries.

SAMPLE INPUT / OUTPUT

```
group_by_initial(['Alice', 'adam', 'Bob', '']) -> {'A': ['Alice', 'adam'], 'B': ['Bob']}
```

30 Student with highest average Medium • ~12 min

Write `top_student(grades)` where `grades` maps student names to non-empty lists of scores. Return the name with highest mean; break ties alphabetically. Return `None` for empty input.

SAMPLE INPUT / OUTPUT

```
top_student({'Zoe': [80, 90], 'Amy': [90, 80]}) -> 'Amy'
top_student({}) -> None
```

CHAPTER 07 / INTERMEDIATE-

Functions & Reuse

Arguments, returns, decomposition, recursion

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

31 Safe division Medium • ~12 min

Write `safe_divide(a, b, default=None)` that returns `a / b` unless b is zero; then return `default`. Demonstrate calling it with a custom default.

SAMPLE INPUT / OUTPUT

```
safe_divide(10, 2) -> 5.0
safe_divide(10, 0, 'N/A') -> 'N/A'
```

32 Count factors Medium • ~12 min

Write `count_factors(n)` that returns the number of positive divisors of positive integer n. Use a helper function if helpful.

SAMPLE INPUT / OUTPUT

```
count_factors(12) -> 6
count_factors(13) -> 2
```

33 Flexible greeting Medium • ~12 min

Write `greet(name, title=None, excited=False)` returning `'Hello, Ada.'`, `'Hello, Dr Ada.'`, or `'Hello, Ada!'` depending on keyword arguments. Title, if supplied, appears before the name.

SAMPLE INPUT / OUTPUT

```
greet('Ada') -> 'Hello, Ada.'
greet('Ada', title='Dr', excited=True) -> 'Hello, Dr Ada!'
```

34 Recursive sum of digits Medium • ~12 min

Write `digit_sum(n)` recursively (a function that calls itself). Sum the digits of a non-negative integer. For zero, return zero.

SAMPLE INPUT / OUTPUT

```
digit_sum(405) -> 9
digit_sum(0) -> 0
```

35 Apply a function to values Medium • ~12 min

Write `apply_to_all(values, transform)` returning a new list after calling the supplied one-argument function for each item. Keep the order; do not modify input.

SAMPLE INPUT / OUTPUT

```
apply_to_all([1, 2, 3], lambda x: x * 2) -> [2, 4, 6]
```

CHAPTER 08 / INTERMEDIATE-

Comprehensions & Sorting

Readable transformations, sorted(), key functions

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

36 Even squares Medium • ~12 min

Write `even_squares(numbers)` returning the squares of only even input numbers in their original order, using one list comprehension.

SAMPLE INPUT / OUTPUT

```
even_squares([1, 2, 3, 4, -6]) -> [4, 16, 36]
```

37 Sort student records Medium • ~12 min

Write `rank_students(students)` for a list of `(name, score)` tuples. Return a new list sorted by descending score and alphabetically ascending name on ties. Do not mutate input.

SAMPLE INPUT / OUTPUT

```
rank_students([('Zoe', 90), ('Amy', 90), ('Ben', 80)]) -> [('Amy', 90), ('Zoe', 90), ('Ben', 80)]
```

38 Invert a unique-value dictionary Medium • ~12 min

Write `invert_dict(mapping)` when all values in the input are unique and hashable. Return a new dictionary in which values become keys and keys become values.

SAMPLE INPUT / OUTPUT

```
invert_dict({'a': 1, 'b': 2}) -> {1: 'a', 2: 'b'}
```

39 Flatten a list of lists Medium • ~12 min

Write `flatten(rows)` flattening one level of nested lists into a new list. Preserve row order and the order within each row. Use a comprehension.

SAMPLE INPUT / OUTPUT

```
flatten([[1, 2], [], [3, 4]]) -> [1, 2, 3, 4]
```

40 Top k most common values Medium • ~12 min

Write `top_k(items, k)` that returns up to k distinct values in descending order of frequency, breaking frequency ties alphabetically. Assume items are strings. `k <= 0` returns `[]`.

SAMPLE INPUT / OUTPUT

```
top_k(['b', 'a', 'b', 'c', 'a'], 2) -> ['a', 'b']
```

CHAPTER 09 / INTERMEDIATE-

Errors, Tests & Defensive Code

Exceptions, assertions, input validation

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

41 Parse an integer safely Medium • ~18 min

Write `parse_int(text)` returning an integer or `None` if the input cannot be parsed by `int()`. Accept leading/trailing whitespace. Catch only the necessary conversion exceptions.

SAMPLE INPUT / OUTPUT

```
parse_int(' 42 ') -> 42
parse_int('4.2') -> None
```

42 Validate a percentage Medium • ~18 min

Write `validate_percent(value)` that returns the numeric value if it is an int or float between 0 and 100 inclusive; otherwise raise `ValueError('percentage must be 0..100')`. Reject booleans even though `bool` subclasses `int`.

SAMPLE INPUT / OUTPUT

```
validate_percent(75) -> 75
validate_percent(120) -> raises ValueError
validate_percent(True) -> raises ValueError
```

43 Fix the mutable default argument bug Medium • ~18 min

The function below unexpectedly shares results between calls. Write a corrected `add_item(item, bucket=None)` that returns a newly created list when no bucket is supplied, but appends to a supplied list in place.

Buggy code: `def add_item(item, bucket=[]): bucket.append(item); return bucket`

SAMPLE INPUT / OUTPUT

```
add_item('a') -> ['a']
add_item('b') -> ['b']
box = []; add_item('x', box); box -> ['x']
```

44 Write assertions for a clamp function Medium • ~18 min

Implement `clamp(value, lower, upper)` that constrains a numeric value to the inclusive range. Assume `lower <= upper`. Then write at least four `assert` tests covering below, inside, above, and boundary values.

SAMPLE INPUT / OUTPUT

```
clamp(-2, 0, 10) -> 0
clamp(15, 0, 10) -> 10
```

45 Normalize mixed numeric data Medium • ~18 min

Write `clean_numbers(values)` that returns a list of floats converted from int, float, or numeric strings; skip `None`, booleans, and invalid strings. Do not raise for unsupported objects.

SAMPLE INPUT / OUTPUT

```
clean_numbers(['10', 2, 'bad', None, 3.5, True]) -> [10.0, 2.0, 3.5]
```

CHAPTER 10 / INTERMEDIATE

Files, CSV & JSON

with, pathlib, csv, json, text processing

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

46 Count non-empty lines in a file Medium • ~18 min

Write `count_nonempty_lines(path)` that reads a UTF-8 text file and counts lines containing non-whitespace characters. Use a context manager. Assume the file exists.

SAMPLE INPUT / OUTPUT

```
File lines: ['hello\n', '   \n', 'world\n'] -> 2
```

47 Save and load JSON settings Medium • ~18 min

Write `save_settings(path, settings)` and `load_settings(path)`. Save the dictionary as UTF-8 JSON with indentation of 2 spaces. Return the decoded Python value from `load_settings`. Assume JSON-compatible data.

SAMPLE INPUT / OUTPUT

```
settings = {'theme': 'dark', 'alerts': True}
save_settings('settings.json', settings)
load_settings('settings.json') -> {'theme': 'dark', 'alerts': True}
```

48 Sum amounts from a CSV Medium • ~18 min

Write `sum_csv_amounts(path)` for a CSV with a header named `amount` and numeric values in that column. Return their sum as a float. Assume a valid UTF-8 CSV file.

SAMPLE INPUT / OUTPUT

```
amount
10.5
20
 -> 30.5
```

49 List .txt files in a directory Medium • ~18 min

Write `list_text_files(folder)` using `pathlib`. Return the names of regular files in that folder (not subfolders) whose extensions are `.txt`, case-insensitive, sorted alphabetically.

SAMPLE INPUT / OUTPUT

```
Folder contains a.txt, B.TXT, data.csv, notes/ -> ['B.TXT', 'a.txt']
```

50 Read a simple access log

Medium • ~18 min

Each UTF-8 log line looks like `2026-10-01 INFO started`. Write `count_log_levels(path)` returning counts by level (the second whitespace-separated token). Ignore blank and malformed lines with fewer than 2 tokens.

SAMPLE INPUT / OUTPUT

```
File:
2026-10-01 INFO started
2026-10-01 ERROR failed
2026-10-01 INFO done
-> {'INFO': 2, 'ERROR': 1}
```

CHAPTER 11 / INTERMEDIATE

Python Classes & OOP

Constructors, methods, encapsulation, properties

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

51 Create a BankAccount Challenge • ~18 min

Implement class `BankAccount` with constructor `__init__(owner, balance=0)`, `deposit(amount)` and `withdraw(amount)`. Both methods require `amount > 0` (otherwise raise `ValueError`). A withdrawal exceeding balance raises `ValueError`. Keep a `balance` attribute.

SAMPLE INPUT / OUTPUT

```
a = BankAccount('Sam', 100)
a.deposit(50); a.withdraw(20)
a.balance -> 130
```

52 Build a Rectangle Challenge • ~18 min

Implement `Rectangle(width, height)` with methods `area()`, `perimeter()` and `is_square()`. Reject non-positive width or height in the constructor with `ValueError`.

SAMPLE INPUT / OUTPUT

```
r = Rectangle(4, 3)
r.area() -> 12
r.perimeter() -> 14
r.is_square() -> False
```

53 Student with an instance method Challenge • ~18 min

Implement `Student(name, scores=None)` and a method `average()` returning the average of scores rounded to 2 decimals, or `None` if there are no scores. Add `add_score(score)` accepting scores 0–100; otherwise raise `ValueError`. Avoid mutable defaults.

SAMPLE INPUT / OUTPUT

```
s = Student('Mia')
s.add_score(80); s.add_score(90)
s.average() -> 85.0
```

54 Protect state with a property Challenge • ~18 min

Implement `Temperature(celsius)` storing the temperature privately as `_celsius`. Expose a read/write `celsius` property. Setter raises `ValueError` below -273.15. Expose read-only `fahrenheit` property.

SAMPLE INPUT / OUTPUT

```
t = Temperature(0)
t.fahrenheit -> 32.0
t.celsius = 100
t.fahrenheit -> 212.0
```

55 ShoppingCart class

Challenge • **~18 min**

Implement `ShoppingCart` that stores `(name, price, quantity)` entries. `add(name, price, quantity=1)` rejects negative price or quantity less than 1. `total()` returns the full sum rounded to 2 decimals. `item_count()` returns total units.

SAMPLE INPUT / OUTPUT

```
cart = ShoppingCart()
cart.add('pen', 10, 2); cart.add('book', 40)
cart.total() -> 60
cart.item_count() -> 3
```

CHAPTER 12 / INTERMEDIATE

Intermediate Patterns & Mini Projects

Dataclasses, inheritance, generators, decorators

WRITE YOUR SOLUTION FIRST. Then try the sample inputs and one original test case.

56 Use inheritance for a savings account

Challenge • ~18 min

Create `SavingsAccount` subclassing the `BankAccount` from exercise 51. Add `apply_interest(rate_percent)` that increases balance by the given percentage; reject negative rates. Inherit deposit and withdrawal unchanged.

SAMPLE INPUT / OUTPUT

```
s = SavingsAccount('Nia', 1000)
s.apply_interest(5)
s.balance -> 1050.0
```

57 Dataclass for an item

Challenge • ~18 min

Create an `Item` dataclass with fields `name: str`, `price: float`, and `quantity: int = 1`, plus method `subtotal()` returning price × quantity. Demonstrate object creation. Assume non-negative values.

SAMPLE INPUT / OUTPUT

```
item = Item('Notebook', 25.0, 3)
item.subtotal() -> 75.0
```

58 Generator for batches

Challenge • ~18 min

Write generator function `batches(items, size)` yielding successive lists of at most `size` elements. If `size <= 0`, raise `ValueError`. Do not build the full list of batches in advance.

SAMPLE INPUT / OUTPUT

```
list(batches([1, 2, 3, 4, 5], 2)) -> [[1, 2], [3, 4], [5]]
```

59 Decorator that counts calls

Challenge • ~18 min

Write `count_calls(func)` decorator that returns a wrapper with `.calls` attribute, initially 0, incremented each time it is called. Preserve support for positional and keyword arguments. No need to preserve function metadata.

SAMPLE INPUT / OUTPUT

```
@count_calls
def double(x): return x * 2
double(3); double(4)
double.calls -> 2
```

60 Mini project: Expense report

Challenge • ~18 min

Write `expense_report(rows)` where `rows` is a list of dictionaries with keys `category` (str) and `amount` (number). Return a dictionary containing `total` (sum rounded to 2 decimals), `by_category` (totals rounded to 2 decimals), and `largest_category` (name with highest total, alphabetical on ties). Empty input returns `{'total': 0.0, 'by_category': {}, 'largest_category': None}`. Do not mutate the rows.

SAMPLE INPUT / OUTPUT

```
expense_report([{'category': 'Food', 'amount': 20}, {'category': 'Travel', 'amount': 50}, {'category': 'Food', 'amount': 30}])
-> {'total': 100.0, 'by_category': {'Food': 50.0, 'Travel': 50.0}, 'largest_category': 'Food'}
```

PART II

WORKED SOLUTIONS

The following pages contain full Python implementations and explanations for exercises 01–60.

Compare only after you have tried each problem. More than one correct approach is possible.

CHAPTER 01 / BEGINNER

Python Basics & Numbers

Variables, arithmetic, type conversion, formatting

WORKED SOLUTIONS

01 Convert Celsius to Fahrenheit

```
def celsius_to_fahrenheit(celsius):
    return round(celsius * 9 / 5 + 32, 1)
```

WHY IT WORKS Multiply first, then add 32. `round(value, 1)` produces the required precision. Arithmetic works for negative temperatures too.

02 Split a restaurant bill

```
def split_bill(total, tip_percent, people):
    with_tip = total * (1 + tip_percent / 100)
    return round(with_tip / people, 2)
```

WHY IT WORKS Convert the percentage into a decimal (`10 / 100 = 0.10`), increase the bill, and divide. Rounding is done last to limit rounding error.

03 Seconds into hours, minutes, seconds

```
def format_duration(seconds):
    hours, remainder = divmod(seconds, 3600)
    minutes, secs = divmod(remainder, 60)
    return f"{hours:02d}:{minutes:02d}:{secs:02d}"
```

WHY IT WORKS `divmod(a, b)` returns quotient and remainder together. First remove full hours; then remove full minutes from the remainder. `:02d` pads to two digits.

04 Calculate compound growth

```
def future_value(principal, annual_rate_percent, years):
    rate = annual_rate_percent / 100
    return round(principal * (1 + rate) ** years, 2)
```

WHY IT WORKS The compound growth formula is P × (1 + r)^n. `**` is exponentiation; `^` is not exponentiation in Python.

05 Calculate a weighted average

```
def weighted_average(scores, weights):
    total = sum(score * weight for score, weight in zip(scores, weights))
    return round(total / sum(weights), 2)
```

WHY IT WORKS `zip()` pairs each score with its matching weight. A generator expression sums the weighted products. Divide by the total weight, not the number of scores.

CHAPTER 02 / BEGINNER

Working with Strings

Slicing, built-in string methods, normalization

WORKED SOLUTIONS

06 Clean a full name

```
def clean_name(name):
    return ' '.join(name.split()).title()
```

WHY IT WORKS `split()` with no argument collapses runs of whitespace, and `' '.join(...)` rebuilds one space between words. `.title()` applies title casing (simplified for names).

07 Reverse words, not letters

```
def reverse_words(sentence):
    words = sentence.split()
    return ' '.join(words[::-1])
```

WHY IT WORKS Slicing `[::-1]` reverses the word list. This preserves each word's characters while reversing their order.

08 Check a relaxed palindrome

```
def is_palindrome(text):
    cleaned = ''.join(char.lower() for char in text if char.isalnum())
    return cleaned == cleaned[::-1]
```

WHY IT WORKS The comprehension keeps letters and digits only. Comparing the normalized string to its reverse makes punctuation and capitalization irrelevant.

09 Count vowels

```
def count_vowels(text):
    return sum(char.lower() in 'aeiou' for char in text)
```

WHY IT WORKS Each membership check is a Boolean; Python treats `True` as 1 and `False` as 0 when summing.

10 Mask an email username

```
def mask_email(email):
    username, domain = email.split('@', 1)
    return username[0] + '*' * (len(username) - 1) + '@' + domain
```

WHY IT WORKS Split once on `@`, then keep the first username character and repeat `*` for the rest. The domain is unchanged.

CHAPTER 03 / BEGINNER

Decisions & Boolean Logic

if / elif / else, comparisons, validation

WORKED SOLUTIONS

11 Classify a number

```
def classify_number(n):
    if n == 0:
        return 'zero'
    sign = 'positive' if n > 0 else 'negative'
    parity = 'even' if n % 2 == 0 else 'odd'
    return f'{sign} {parity}'
```

WHY IT WORKS Handle zero first. `% 2` tests evenness, including for negative integers. Then combine two independent classifications.

12 Leap year rule

```
def is_leap_year(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
```

WHY IT WORKS Century years are a special case. A year divisible by 400 qualifies; otherwise it must divide by 4 but not by 100.

13 Shipping fee bands

```
def shipping_fee(order_total):
    if order_total >= 1500:
        return 0
    if order_total >= 500:
        return 80
    return 150
```

WHY IT WORKS Testing from highest threshold downward avoids repeated upper-bound conditions. An early `return` ends the function.

14 Validate a triangle

```
def triangle_type(a, b, c):
    if min(a, b, c) <= 0 or a + b <= c or a + c <= b or b + c <= a:
        return 'invalid'
    if a == b == c:
        return 'equilateral'
    if a == b or b == c or a == c:
        return 'isosceles'
    return 'scalene'
```

WHY IT WORKS Check validity before comparing equal sides: three equal negative numbers are not a triangle. Strict inequality excludes degenerate (flat) triangles.

15 Grade a score

```
def grade(score):
    if not 0 <= score <= 100:
        return 'Invalid'
    if score >= 90:
        return 'A'
    if score >= 80:
        return 'B'
    if score >= 70:
        return 'C'
    if score >= 60:
        return 'D'
    return 'F'
```

WHY IT WORKS Validate the range first. Ordered comparisons mean `score >= 80` automatically implies the score is below 90 after the earlier return.

CHAPTER 04 / BEGINNER

Loops & Number Patterns

for, while, range, simple algorithms

WORKED SOLUTIONS

16 Sum multiples of 3 or 5

```
def sum_multiples(limit):
    total = 0
    for number in range(1, limit):
        if number % 3 == 0 or number % 5 == 0:
            total += number
    return total
```

WHY IT WORKS Iterate from 1 to `limit - 1`. Use `or` so shared multiples such as 15 are counted once.

17 Factorial with a loop

```
def factorial(n):
    result = 1
    for number in range(2, n + 1):
        result *= number
    return result
```

WHY IT WORKS Start with the multiplicative identity, 1. The empty loop for `n = 0` naturally returns 1.

18 FizzBuzz list

```
def fizzbuzz(n):
    result = []
    for number in range(1, n + 1):
        if number % 15 == 0:
            result.append('FizzBuzz')
        elif number % 3 == 0:
            result.append('Fizz')
        elif number % 5 == 0:
            result.append('Buzz')
        else:
            result.append(number)
    return result
```

WHY IT WORKS Check both divisors before either individual case. Otherwise 15 would be labelled only `Fizz` or `Buzz`.

19 Count digits without strings

```
def digit_count(n):
    n = abs(n)
    if n == 0:
        return 1
    count = 0
    while n > 0:
        n //= 10
        count += 1
    return count
```

WHY IT WORKS Each integer division by 10 removes the rightmost digit. `abs()` discards the sign, and zero needs special handling.

20 Find all primes up to n

```
def primes_up_to(n):
    primes = []
    for number in range(2, n + 1):
        is_prime = True
        for divisor in range(2, int(number ** 0.5) + 1):
            if number % divisor == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(number)
    return primes
```

WHY IT WORKS Only divisors up to the square root need checking: if a larger divisor exists, its matching factor is smaller. `break` stops as soon as a factor is found.

CHAPTER 05 / BEGINNER+

Lists & Tuples

Indexing, mutation, traversal, nested lists

WORKED SOLUTIONS

21 Remove duplicates, keep order

```
def unique_in_order(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

WHY IT WORKS A set gives fast membership checks; a list preserves the order of first appearances. `list(set(items))` would not guarantee the required order.

22 Second distinct largest

```
def second_largest(values):
    distinct = sorted(set(values), reverse=True)
    return distinct[1] if len(distinct) >= 2 else None
```

WHY IT WORKS Convert to a set to remove duplicates; sort descending so index 1 is the second distinct largest. Handle the short list before indexing.

23 Rotate a list right

```
def rotate_right(items, k):
    if not items:
        return []
    k %= len(items)
    return items[-k:] + items[:-k] if k else items[:]
```

WHY IT WORKS Reduce k using modulo so a rotation of the list length has no effect. Slicing combines the final k items with the rest. The k=0 branch avoids `items[-0:]` (the entire list).

24 Running totals

```
def running_totals(numbers):
    total = 0
    result = []
    for number in numbers:
        total += number
        result.append(total)
    return result
```

WHY IT WORKS Keep a single accumulator and append after each addition. This runs in O(n) time and does not repeatedly sum prefixes.

25 Transpose a rectangular matrix

```
def transpose(matrix):
    if not matrix or not matrix[0]:
        return []
    return [list(column) for column in zip(*matrix)]
```

WHY IT WORKS `*matrix` unpacks its rows as arguments to `zip`, which groups elements at the same index. Convert resulting tuples to lists.

CHAPTER 06 / BEGINNER+

Dictionaries & Sets

Counting, grouping, keys, membership

WORKED SOLUTIONS

26 Word frequency counter

```
def word_counts(text):
    counts = {}
    for token in text.lower().split():
        word = token.strip('.,!?;:')
        if word:
            counts[word] = counts.get(word, 0) + 1
    return counts
```

WHY IT WORKS `dict.get(key, 0)` provides zero for unseen words. Normalization must happen before counting so `Hi,` and `hi!` share a key.

27 Merge inventory counts

```
def merge_inventory(left, right):
    result = left.copy()
    for product, quantity in right.items():
        result[product] = result.get(product, 0) + quantity
    return result
```

WHY IT WORKS Copy the first dictionary to prevent accidental mutation, then add quantities from the second with `.get()`.

28 Common unique values

```
def common_values(a, b):
    return sorted(set(a) & set(b))
```

WHY IT WORKS Set intersection (`&`) finds common members and automatically removes duplicates. Sorting makes the output deterministic.

29 Group names by initial

```
def group_by_initial(names):
    groups = {}
    for name in names:
        if not name.strip():
            continue
        key = name.strip()[0].upper()
        groups.setdefault(key, []).append(name)
    return groups
```

WHY IT WORKS `setdefault(key, [])` creates the list the first time an initial appears, then returns it. Appending while traversing preserves arrival order.

30 Student with highest average

```
def top_student(grades):
    if not grades:
        return None
    return min(grades, key=lambda name: (-sum(grades[name]) / len(grades[name]), name))
```

WHY IT WORKS `min()` with a tuple key first favors the most negative average (the highest actual average), then the alphabetically smallest name to break ties.

CHAPTER 07 / INTERMEDIATE-

Functions & Reuse

Arguments, returns, decomposition, recursion

WORKED SOLUTIONS

31 Safe division

```
def safe_divide(a, b, default=None):
    if b == 0:
        return default
    return a / b
```

WHY IT WORKS A default argument makes the fallback optional. Branching before division avoids `ZeroDivisionError`. Returning `None` can be a deliberate API choice.

32 Count factors

```
def count_factors(n):
    count = 0
    for divisor in range(1, int(n ** 0.5) + 1):
        if n % divisor == 0:
            count += 1 if divisor * divisor == n else 2
    return count
```

WHY IT WORKS Divisors occur in pairs `d` and `n//d`; only the square root is unpaired for perfect squares. This avoids a full scan from 1 through n.

33 Flexible greeting

```
def greet(name, title=None, excited=False):
    full_name = f'{title} {name}' if title else name
    punctuation = '!' if excited else '.'
    return f'Hello, {full_name}{punctuation}'
```

WHY IT WORKS Keyword defaults support multiple forms without overloaded functions. Build the name and punctuation separately to keep the function readable.

34 Recursive sum of digits

```
def digit_sum(n):
    if n < 10:
        return n
    return n % 10 + digit_sum(n // 10)
```

WHY IT WORKS The base case is a single digit. Each recursive call removes the last digit with `// 10`, while `% 10` adds that digit to the total.

35 Apply a function to values

```
def apply_to_all(values, transform):
    return [transform(value) for value in values]
```

WHY IT WORKS Functions are first-class objects: pass them as values. Each item is transformed by calling `transform(value)`. A list comprehension returns a fresh list.

CHAPTER 08 / INTERMEDIATE-

Comprehensions & Sorting

Readable transformations, sorted(), key functions

WORKED SOLUTIONS

36 Even squares

```
def even_squares(numbers):
    return [n * n for n in numbers if n % 2 == 0]
```

WHY IT WORKS The comprehension's trailing `if` filters, and the expression on the left transforms. Order is retained.

37 Sort student records

```
def rank_students(students):
    return sorted(students, key=lambda pair: (-pair[1], pair[0]))
```

WHY IT WORKS The key tuple first negates the score to get a descending numeric order, then uses the name ascending. `sorted()` does not mutate the input.

38 Invert a unique-value dictionary

```
def invert_dict(mapping):
    return {value: key for key, value in mapping.items()}
```

WHY IT WORKS A dictionary comprehension swaps each `(key, value)` pair. Unique input values prevent overwriting duplicate inverted keys.

39 Flatten a list of lists

```
def flatten(rows):
    return [item for row in rows for item in row]
```

WHY IT WORKS Nested `for` clauses read left to right, like nested loops: visit each row, then each item in that row.

40 Top k most common values

```
def top_k(items, k):
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    ordered = sorted(counts, key=lambda item: (-counts[item], item))
    return ordered[:max(k, 0)]
```

WHY IT WORKS Count items first, then sort distinct keys using a two-part sort key. The slice selects the requested number and safely returns an empty list for non-positive k.

CHAPTER 09 / INTERMEDIATE-

Errors, Tests & Defensive Code

Exceptions, assertions, input validation

WORKED SOLUTIONS

41 Parse an integer safely

```
def parse_int(text):
    try:
        return int(text)
    except (ValueError, TypeError):
        return None
```

WHY IT WORKS `int()` accepts whitespace but rejects decimals written as text. Catch expected conversion failures, rather than hiding every programming error with a bare `except`.

42 Validate a percentage

```
def validate_percent(value):
    if isinstance(value, bool) or not isinstance(value, (int, float)) or not 0 <= value <= 100:
        raise ValueError('percentage must be 0..100')
    return value
```

WHY IT WORKS Explicit validation at function boundaries produces predictable failures. `isinstance(True, int)` is true, so reject bool first.

43 Fix the mutable default argument bug

```
def add_item(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

WHY IT WORKS Python evaluates a default list only once at function definition time. Use `None` as a sentinel and allocate a fresh list inside the call.

44 Write assertions for a clamp function

```
def clamp(value, lower, upper):
    return max(lower, min(value, upper))

assert clamp(-2, 0, 10) == 0
assert clamp(5, 0, 10) == 5
assert clamp(15, 0, 10) == 10
assert clamp(10, 0, 10) == 10
```

WHY IT WORKS The inner `min` caps the top; the outer `max` ensures the bottom. `assert` checks assumptions during learning (production tests normally use a test framework).

45 Normalize mixed numeric data

```
def clean_numbers(values):
    cleaned = []
    for value in values:
        if value is None or isinstance(value, bool):
            continue
        try:
            cleaned.append(float(value))
        except (ValueError, TypeError):
            continue
    return cleaned
```

WHY IT WORKS A `try` block captures expected conversion failures, while early checks reject `None` and booleans. Returning a fresh list avoids mutating inputs.

CHAPTER 10 / INTERMEDIATE

Files, CSV & JSON

with, pathlib, csv, json, text processing

WORKED SOLUTIONS

46 Count non-empty lines in a file

```
def count_nonempty_lines(path):
    with open(path, 'r', encoding='utf-8') as file:
        return sum(bool(line.strip()) for line in file)
```

WHY IT WORKS `with` closes the file automatically, including after errors. `.strip()` converts whitespace-only lines to empty strings.

47 Save and load JSON settings

```
import json

def save_settings(path, settings):
    with open(path, 'w', encoding='utf-8') as file:
        json.dump(settings, file, indent=2)

def load_settings(path):
    with open(path, 'r', encoding='utf-8') as file:
        return json.load(file)
```

WHY IT WORKS `json.dump` writes Python data directly to a file; `json.load` parses it back. JSON booleans become Python booleans on loading.

48 Sum amounts from a CSV

```
import csv

def sum_csv_amounts(path):
    total = 0.0
    with open(path, 'r', encoding='utf-8', newline='') as file:
        for row in csv.DictReader(file):
            total += float(row['amount'])
    return total
```

WHY IT WORKS `csv.DictReader` accesses columns by name, avoiding fragile column positions. `newline=''` is recommended for CSV handling.

49 List .txt files in a directory

```
from pathlib import Path

def list_text_files(folder):
    directory = Path(folder)
    names = [p.name for p in directory.iterdir() if p.is_file() and p.suffix.lower() == '.txt']
    return sorted(names)
```

WHY IT WORKS `Path.iterdir()` yields direct children; `is_file()` excludes folders. Lowercasing only the extension makes matching case-insensitive.

50 Read a simple access log

```
def count_log_levels(path):
    counts = {}
    with open(path, 'r', encoding='utf-8') as file:
        for line in file:
            parts = line.split()
            if len(parts) < 2:
                continue
            level = parts[1]
            counts[level] = counts.get(level, 0) + 1
    return counts
```

WHY IT WORKS Split each record into whitespace-separated tokens. Skip incomplete rows and count the level field using a dictionary.

CHAPTER 11 / INTERMEDIATE

Python Classes & OOP

Constructors, methods, encapsulation, properties

WORKED SOLUTIONS

51 Create a BankAccount

```
class BankAccount:
    def __init__(self, owner, balance=0):
        if balance < 0:
            raise ValueError('initial balance cannot be negative')
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError('amount must be positive')
        self.balance += amount

    def withdraw(self, amount):
        if amount <= 0 or amount > self.balance:
            raise ValueError('invalid withdrawal')
        self.balance -= amount
```

WHY IT WORKS `__init__` initializes each new instance. `self` refers to that instance. Methods both validate inputs and change instance state through `self.balance`.

52 Build a Rectangle

```
class Rectangle:
    def __init__(self, width, height):
        if width <= 0 or height <= 0:
            raise ValueError('dimensions must be positive')
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def is_square(self):
        return self.width == self.height
```

WHY IT WORKS Store dimensions as instance attributes and expose related calculations as methods. A method can return a Boolean just like a regular function.

53 Student with an instance method

```
class Student:
    def __init__(self, name, scores=None):
        self.name = name
        self.scores = list(scores) if scores is not None else []

    def add_score(self, score):
        if not 0 <= score <= 100:
            raise ValueError('score must be 0..100')
        self.scores.append(score)

    def average(self):
        if not self.scores:
            return None
        return round(sum(self.scores) / len(self.scores), 2)
```

WHY IT WORKS Create a new list per student, copying supplied lists as well. Instance methods use `self.scores` to access the object's state.

54 Protect state with a property

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError('below absolute zero')
        self._celsius = value

    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32
```

WHY IT WORKS `@property` lets callers use attribute syntax while running validation or calculations. The constructor delegates to the setter, keeping one validation rule.

55 ShoppingCart class

```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, name, price, quantity=1):
        if price < 0 or quantity < 1:
            raise ValueError('invalid price or quantity')
        self.items.append((name, price, quantity))

    def total(self):
        return round(sum(price * quantity for _, price, quantity in self.items), 2)

    def item_count(self):
        return sum(quantity for _, _, quantity in self.items)
```

WHY IT WORKS An object's list accumulates entries across method calls. Tuple unpacking makes totals simple. This intentionally permits separate entries with the same name.

CHAPTER 12 / INTERMEDIATE

Intermediate Patterns & Mini Projects

Dataclasses, inheritance, generators, decorators

WORKED SOLUTIONS

56 Use inheritance for a savings account

```
class SavingsAccount(BankAccount):
    def apply_interest(self, rate_percent):
        if rate_percent < 0:
            raise ValueError('rate cannot be negative')
        self.balance *= 1 + rate_percent / 100
```

WHY IT WORKS Inheritance reuses the parent's constructor and other methods. A subclass adds new behavior without duplicating validation logic. Run exercise 51's class definition first.

57 Dataclass for an item

```
from dataclasses import dataclass

@dataclass
class Item:
    name: str
    price: float
    quantity: int = 1

    def subtotal(self):
        return self.price * self.quantity
```

WHY IT WORKS `@dataclass` generates common boilerplate such as `__init__` and a helpful representation. You still write your own methods for domain behavior.

58 Generator for batches

```
def batches(items, size):
    if size <= 0:
        raise ValueError('size must be positive')
    for start in range(0, len(items), size):
        yield items[start:start + size]
```

WHY IT WORKS `yield` pauses the function and produces one batch at a time. A range with step `size` advances through non-overlapping chunks.

59 Decorator that counts calls

```
def count_calls(func):
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper
```

WHY IT WORKS The decorator replaces a function with a wrapper. `*args` and `**kwargs` forward arbitrary arguments. The wrapper stores its call counter as a function attribute.

60 Mini project: Expense report

```
def expense_report(rows):
    totals = {}
    for row in rows:
        category = row['category']
        totals[category] = totals.get(category, 0.0) + row['amount']
    rounded = {name: round(amount, 2) for name, amount in totals.items()}
    largest = min(totals, key=lambda name: (-totals[name], name)) if totals else None
    return {
        'total': round(sum(totals.values()), 2),
        'by_category': rounded,
        'largest_category': largest,
    }
```

WHY IT WORKS This combines dictionaries, loops, comprehensions, sorting keys, and careful output design. Accumulate before rounding, and use a tuple key to break ties by category name.

END OF WORKBOOK • NEXT STEP: REVISIT EVERY QUESTION YOU COULD NOT SOLVE UNAIDED.